

Multi-Level Phishing URL Risk Scoring Using Explainable AI

A Leakage-Aware Study of Generalization and Dataset Shift

Md Shariful Islam Shahed (0112230392) Tasin Khan Orith (0112230393) Rohan Hasan (0112230148)
Mushfiq Labib Maher (011222343) Mahadin Islam (0112230380) **Section: B**

Department of Computer Science and Engineering, United International University

Submitted to: **Mr. Asif Abrar**, Lecturer, Dept. of CSE, United International University

Code repository: <https://github.com/SIShahed16261/ML-Phishing-Link-Scoring>

Abstract

Phishing URLs remain a leading entry point for credential theft, and blacklists cannot keep pace with new domains. We present an end-to-end, URL-only phishing detection system that outputs a binary decision, a phishing probability, a five-level risk score (SAFE to CRITICAL), and a SHAP-based explanation. Our emphasis is methodological validity rather than headline accuracy. We merge the Majestic Million and Malicious URLs datasets into a corpus of 1,510,543 URLs (6.1% phishing), apply a domain-grouped train/validation/test split with zero domain overlap, rebalance only the training set, and extract 63 lexical features that are reduced to 47 without touching the test set. Five learners are compared under a bias-variance lens: Logistic Regression, Random Forest, XGBoost, LightGBM and CatBoost. XGBoost is selected on validation data and reaches 86.89% accuracy, 0.8965 ROC-AUC, 79.79% phishing recall and 0.548 average precision on an untouched, naturally imbalanced test set of 274,193 URLs, with a 2.1-point train-validation accuracy gap. The risk levels are monotone in empirical phishing rate (0.9% in SAFE to 44.5% in CRITICAL). External validation on PhiUSIIL, however, exposes severe dataset shift: with the correct label mapping, the model flags 96.3% of all PhiUSIIL URLs as phishing and its ROC-AUC falls to 0.344. A feature-level analysis traces this to source artefacts, chiefly HTTPS and www prefixes that never occur among training benign URLs. Such shortcut learning, not in-distribution accuracy, is the central open problem for lexical detectors.

Keywords: phishing detection, URL lexical features, explainable AI, SHAP, domain-aware splitting, bias-variance trade-off, dataset shift, gradient boosting.

1 Introduction

1.1 Background

Phishing is a social-engineering attack in which an adversary imitates a trusted entity, typically through a link to a counterfeit web page, to steal credentials or payment data. The Anti-Phishing Working Group reports hundreds of thousands to over a million unique phishing attacks per quarter [1]. Because phishing domains often live for only hours, blacklists are inherently reactive. Classifiers that inspect only the URL string can score a link *before* it is visited, need no page download and run in milliseconds, which makes them attractive for browsers, e-mail gateways and messaging platforms [2, 3].

1.2 Motivation

Lexical phishing detectors frequently report accuracies above 95% [4], but two problems make such numbers hard to trust. First, random row-level splitting lets URLs from the same domain appear in both training and test sets, so a model can succeed by memorising domains—a classic data leakage [5]. Second, benign and phishing URLs usually come from *different sources*, so a classifier may learn to recognise the source instead of the attack [6, 7]. Analysts also need to know *why* a link was flagged. We therefore aim for an evaluation that is leakage-aware, tested on an independent dataset, and explained.

1.3 Research Objective and Contributions

Our objective is to build a URL-only phishing risk scorer and to study, honestly, how well it generalises. Our contributions are:

1. **A complete, reproducible pipeline** (Fig. 1) from raw URL to probability, 0–100 risk score, five-level risk category and SHAP explanation.

2. **Leakage-aware evaluation:** a domain-grouped split with verified zero domain overlap across 999,200 unique domains, and class rebalancing confined to the training set so that validation and test sets keep the real $\approx 6\%$ phishing prevalence.
3. **A bias-variance comparison of five learners** spanning linear, bagging and three boosting paradigms, including an analysis of why the accuracy gap is confounded by the change of class prior between training and validation.
4. **Global and local explanations** with SHAP, and an empirical check that risk levels are monotone in observed phishing rate.
5. **A cross-dataset study on PhiUSIIL** that corrects a label-polarity pitfall, quantifies the performance collapse, and attributes it to specific source artefacts via feature-distribution analysis.

2 Related Works

Lexical and host-based classification. Garera et al. [8] showed that structural cues (obfuscated hosts, IP addresses, suspicious tokens) separate phishing from benign URLs with logistic regression. Ma et al. [2] combined bag-of-words lexical features with WHOIS and DNS features to go “beyond blacklists”. Verma and Das [9] emphasised fast extraction of character n -gram and entropy features. Sahingoz et al. [4] used NLP-derived word features and reported 97.98% accuracy with Random Forest, on a random split without domain control.

Deep learning. URLNet [10] learns character- and word-level CNN embeddings, and Aljofey et al. [11] proposed a character-level CNN. Such models remove manual feature design but need large corpora and are harder to explain.

Ensembles, datasets and explainability. Random Forest [15] and gradient-boosted trees—XGBoost [12], LightGBM [13], CatBoost [14]—are strong defaults for tabular features. PhiUSIIL [16] provides 235,795 URLs with similarity-index and page-content features. SHAP [17] attributes each prediction additively to features. Hannousse and Yahiouche [6] showed that phishing benchmarks differ widely in construction and that results do not transfer reliably between them, echoing general findings on leakage [5], dataset shift [18] and shortcut learning [7]. Table 1 compares representative works.

Table 1: Comparison of representative URL-based phishing detection studies with this work.

Work	Features	Model(s)	Domain split	XAI	Ext. test
Garera et al. [8]	URL structure	LR	–	–	–
Ma et al. [2]	Lexical + host (WHOIS/DNS)	LR, SVM, NB	–	–	–
Verma & Das [9]	Char. n -grams, entropy	Multiple	–	–	–
Le et al. [10]	Learned char/word embeddings	CNN	–	–	–
Sahingoz et al. [4]	NLP word features	RF + 6 others	–	–	–
Aljofey et al. [11]	Character sequence	CNN	–	–	–
Prasad & Chandra [16]	URL + page content	Incremental learners	–	–	–
This work	47 lexical	LR, RF, 3 GBDTs	✓	SHAP	PhiUSIIL

3 Gap Analysis

From the literature we identify four gaps:

1. **Leakage-prone evaluation.** Most studies split at URL level although popular domains contribute thousands of URLs (13,430 from wikipedia.org in our corpus); domain-disjoint splits are rarely enforced or verified.
2. **Unrealistic test balance.** Balanced test sets inflate precision and hide the false-alarm burden of a $\approx 6\%$ real prevalence.
3. **Accuracy without generalisation analysis.** Train-validation gaps and performance on independently collected data are seldom reported.
4. **Binary output without graded, explained risk.** Analysts need a graded score whose levels are shown to be meaningful, together with per-URL reasons.

We address (1)–(2) with domain-grouped splitting and train-only rebalancing, (3) with a five-model bias-variance comparison and PhiUSIIL shift analysis, and (4) with a five-level risk score and SHAP.

4 Methodology

Figure 1 summarises the pipeline. The offline stage builds and validates the model; the online stage scores a single URL.

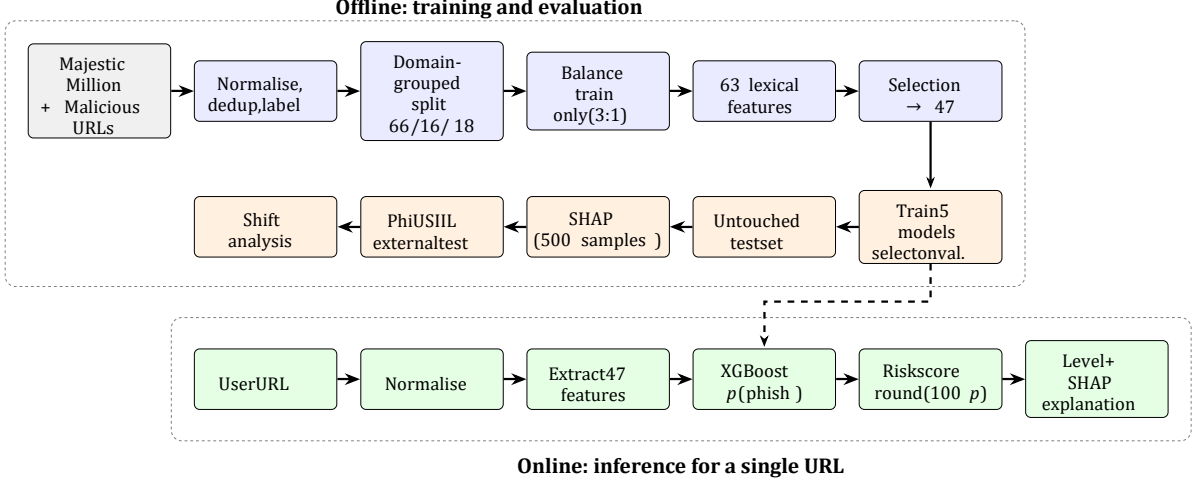


Figure 1: System architecture. The offline stage (top) builds, selects and audits the model; the online stage (bottom) turns any URL into a probability, a 0–100 risk score, a risk level and a SHAP explanation.

4.1 Dataset Description

We use three public datasets (Fig. 2a):

Majestic Million [19]: the top 1,000,000 domains ranked by referring subnets. All entries are treated as benign; they are bare domains (no scheme, no path).

Malicious URLs Dataset [20]: 651,191 URLs labelled benign (428,103), defacement (96,457), phishing (94,111) and malware (32,520). We keep only benign and phishing to keep the task focused, giving 522,214 URLs.

PhiUSIIL [16]: 235,795 URLs (134,850 legitimate, 100,945 phishing) with 55 columns. It is held out entirely for external validation. Note that in PhiUSIIL *label 1 denotes a legitimate URL and label 0 a phishing URL*; we verified this by inspection (label-1 rows are exclusively <https://www.home> pages with page titles, while label-0 rows include IPFS, firebaseapp and look-alike login pages).

Merging the first two sources gives 1,522,214 URLs; after normalisation and removal of 11,671 duplicates the development corpus contains **1,510,543 URLs**: 1,418,416 benign and 92,127 phishing (6.1%, Fig. 2b).

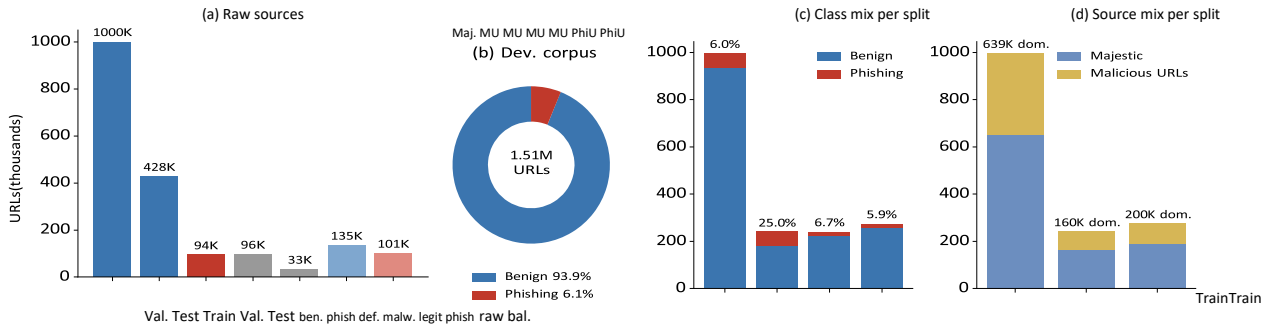


Figure 2: Data overview. (a) Size and label of each raw source; grey bars are excluded and PhiUSIIL is used only for external validation. (b) Class distribution of the merged development corpus (15:1 imbalance). (c) Benign/phishing counts and phishing share per split; only the training set is rebalanced. (d) Source mix and number of unique, disjoint domains per split.

4.2 Data Preprocessing

URL normalisation. Each URL is stripped of whitespace, given an `http://` scheme if none is present, lower-cased, parsed, stripped of a leading `www.` and of a trailing slash, and reassembled with its query and fragment. This yields a canonical form so that, for example, `https://www.google.com/` and `google.com` are recognised as duplicates.

Domain extraction. The registrable domain is taken as the last two labels of the host (or the full host for IP addresses), giving 999,200 unique domains (1.51 URLs per domain on average). 8,453 domains contain both benign and phishing URLs (e.g., `000webhostapp.com`: 1 benign, 1,189 phishing), which illustrates why domain memorisation would inflate scores.

Domain-aware splitting. We apply `GroupShuffleSplit` with domains as groups (seed 42): 20% of domains form the test set and 20% of the remaining domains form the validation set. Because domains differ in size, this yields 996,327 training (66.0%), 240,023 validation (15.9%) and 274,193 test (18.2%) URLs. Set intersections of the domain lists are empty for all three pairs, confirming zero leakage. Phishing prevalence is 6.0%, 6.7% and 5.9% respectively (Fig. 2c–d).

Class balancing. Only the training set is rebalanced: benign URLs are randomly undersampled to three times the phishing count, giving 179,580 benign and 59,860 phishing URLs (25% phishing, 239,440 in total). Validation and test sets are left at their natural prevalence so that precision and false-positive rates reflect deployment conditions. Boosting models additionally use `scale_pos_weight=3` and Logistic Regression and Random Forest use `class_weight='balanced'`.

4.3 Feature Engineering and Selection

We extract 63 lexical features, grouped into five categories (Table 2). All are computed from the URL string alone, so no network request is needed at inference time.

Table 2: Lexical features extracted from each normalised URL (63 in total before selection).

Category	#	Features
URL structure	9	URL, hostname, path, query and fragment length; number of path segments, query parameters and subdomains; path+query+fragment length
Character counts	19	Counts of <code>.</code> <code>/</code> <code>-</code> <code>_</code> <code>?</code> <code>=</code> <code>&</code> <code>%</code> <code>@</code> <code>:</code> <code>;</code> ; number of letters, digits and special characters; letter, digit and special-character ratios; digit-to-letter ratio; longest run of special characters
Security indicators	7	HTTPS scheme; IP-address host; punycode (<code>xn--</code>); suspicious TLD (17 TLDs such as <code>.tk</code> , <code>.xyz</code> , <code>.top</code>); percent-encoding; <code>>4</code> subdomains; presence of <code>//</code>
Token & entropy	8	Number of alphanumeric tokens; mean, max and min token length; token diversity; Shannon entropy of URL, hostname and path
Phishing keywords	20	Counts of 18 keywords (<i>login, signin, verify, verification, secure, account, update, confirm, password, bank, payment, authenticate, security, validate, authorize, alert, warning, suspicious</i>); total keyword count; any-keyword flag

Entropy is $H(s) = -\sum_c p_c \log_2 p_c$ over character frequencies p_c ; randomised phishing-kit paths tend to have higher entropy.

Feature selection uses the training set only (Fig. 3a): (i) 2 constant features are dropped (`has_double_slash` is always 1 after normalisation adds a scheme; `keyword_suspicious` is always 0); (ii) 8 features with variance below 10^{-3} are dropped (mostly rare keywords); (iii) of every pair with Pearson $|r| > 0.95$ one is dropped, removing `path_query_fragment_length`, `slash_count`, `equal_sign_count`, `num_letters`, `digit_to_letter_ratio` and `num_tokens`. The final set has **47 features**. Fig. 3b shows Random Forest impurity importance on this set, and Fig. 3c shows the ten features with the largest benign–phishing mean difference: phishing URLs are on average longer (48.1 vs. 34.5 characters) and have roughly twice the path length.

4.4 Models

We compare five learners that span the bias–variance spectrum (Table 3). Logistic Regression (LR) is a high-bias linear baseline, trained on standardised features. Random Forest (RF) [15] reduces variance by bagging deep trees. XGBoost [12], LightGBM [13] and CatBoost [14] are gradient-boosted tree ensembles that reduce bias sequentially and control variance through shrinkage, subsampling and L_1/L_2 penalties. Hyperparameters were fixed to moderate, commonly used values and were not tuned on the test set.

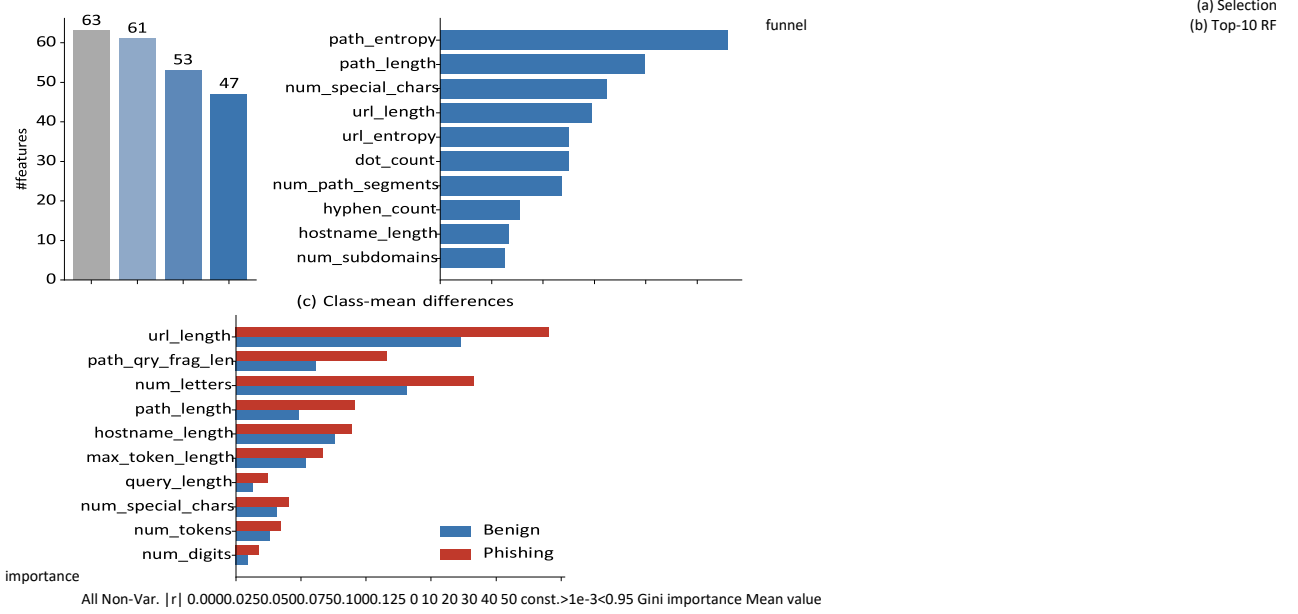


Figure 3: (a) Features remaining after each selection step. (b) Top-10 Random Forest (Gini) importances on the selected 47 features. (c) Mean benign vs. phishing values (balanced training set) for the ten features with the largest absolute difference.

Table 3: Model configurations. All models use random seed 42.

Model	Paradigm	Key hyperparameters
Logistic Regression	Linear	$C = 1.0$, L-BFGS, max iter = 1000, class weight = balanced, standardised inputs
Random Forest	Bagging	150 trees, max depth = 15, min samples split = 10, min samples leaf = 5, balanced
XGBoost	Boosting	150 trees, depth 6, $\eta = 0.1$, subsample = 0.8, colsample = 0.8, $\alpha = 0.1$, $\lambda = 1$, spw = 3
LightGBM	Boosting	150 trees, depth 6, $\eta = 0.1$, subsample = 0.8, colsample = 0.8, $\alpha = 0.1$, $\lambda = 1$, spw = 3
CatBoost	Boosting	150 iterations, depth 6, $\eta = 0.1$, scale pos weight = 3

4.5 Risk Scoring and Explainability

The selected model outputs $p = P(\text{phishing} \mid \mathbf{x})$. The risk score is $R = \text{round}(100p) \in [0, 100]$ and is mapped to five levels: SAFE ($R \leq 20$), LOW (21–40), MEDIUM (41–60), HIGH (61–80) and CRITICAL ($R > 80$).

The binary decision uses $p \geq 0.5$.

For explanation we use TreeSHAP [17], which computes exact Shapley values for tree ensembles. For a prediction $f(\mathbf{x})$ in log-odds space, SHAP gives $f(\mathbf{x}) = \phi_0 + \sum_{j=1}^{47} \phi_j$, where ϕ_0 is the expected model output and ϕ_j is the contribution of feature j . Global importance is $\frac{1}{N} \sum_i |\phi_j^{(i)}|$ over $N = 500$ test URLs, a subset chosen to keep computation tractable.

4.6 Training Procedure and Evaluation Metrics

All models are trained on the balanced training set. Model selection uses the validation set only; the test set is used exactly once for the selected model. We report accuracy, precision, recall, F1 for the phishing class, ROC-AUC, average precision (area under the precision–recall curve), false-positive rate (FPR) and false-negative rate (FNR). Because accuracy is dominated by the majority class at 6% prevalence, we treat ROC-AUC and PR metrics as primary [21] and additionally report balanced accuracy and the Matthews correlation coefficient (MCC) [22].

Generalisation is measured by the train-validation gap in accuracy, F1 and AUC. Experiments ran on Kaggle (Python 3.12, scikit-learn [23], XGBoost, LightGBM, CatBoost, SHAP 0.51).

5 Results and Discussion

5.1 Model Comparison

Table 4 and Fig. 4 compare the five models. The four tree ensembles are tightly clustered on validation data (AUC 0.8955–0.8975, F1 0.4165–0.4221), while LR is clearly weaker (AUC 0.8632, F1 0.3753). XGBoost is best on all three validation metrics and trains in under 3s, so it is selected as the final model. The small spread among ensembles suggests that performance is limited by the information in lexical features rather than by the learner.

Table 4: Training and validation performance of the five models. Gap = train – validation. Best validation values in bold.

Model	Accuracy			F1(phishing)		ROC-AUC			Time (s)
	Train	Val	Gap	Train	Val	Train	Val	Gap	
Logistic Regression	0.8021	0.8284	−0.0263	0.6599	0.3753	0.8509	0.8632	−0.0123	45.40
Random Forest	0.8777	0.8494	0.0283	0.7747	0.4214	0.9240	0.8971	0.0269	19.48
XGBoost	0.8726	0.8512	0.0214	0.7639	0.4221	0.9185	0.8975	0.0210	2.83
LightGBM	0.8680	0.8488	0.0192	0.7571	0.4184	0.9156	0.8961	0.0195	2.83
CatBoost	0.8616	0.8466	0.0150	0.7477	0.4165	0.9096	0.8955	0.0141	6.62

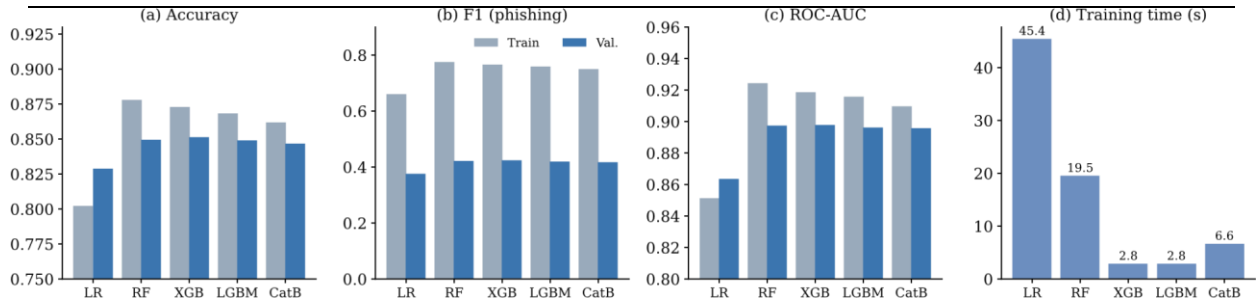


Figure 4: (a)–(c) Train (grey) and validation (blue) accuracy, phishing F1 and ROC-AUC per model (note the different y-axis ranges; the F1 drop is mainly caused by the change in class prior). (d) Wall-clock training time on 239,440 samples; the histogram-based boosters are an order of magnitude faster than RF and LR.

5.2 Bias-Variance Analysis

Fig. 5 visualises the train-validation gaps. Among the ensembles, the gap follows model flexibility: RF, whose trees reach depth 15, has the largest AUC gap (0.027), the depth-6 boosters are smaller (0.020–0.021 for XGBoost and LightGBM) and CatBoost, whose symmetric trees are more constrained, is the smallest (0.014). No ensemble shows a large gap, so none is severely overfitting. LR has the lowest training scores of all models, the signature of high bias.

Two cautions apply. First, training has 25% phishing but validation 6.7%. With fewer positives the same false-positive rate yields far more false alarms per detection, so precision and F1 drop sharply (XGBoost F1: 0.76 → 0.42); this prior shift, not overfitting, drives the large F1 gap and also explains LR’s *negative* accuracy gap, since a benign-heavy validation set rewards a model that is relatively good on benign URLs. ROC-AUC is prior-invariant, so its gap is the cleanest variance measure here. Second, the gap measures generalisation only *within* the development distribution (see Section 5.6).

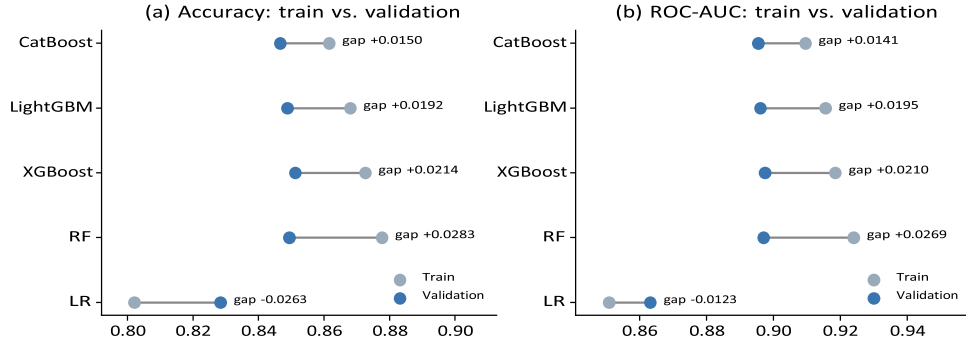


Figure 5: Train-validation comparison for (a) accuracy and (b) ROC-AUC. Longer connectors indicate larger gaps (higher variance); lower training scores indicate higher bias. LR is left-most (high bias), RF has the widest ensemble gap.

5.3 Final Evaluation on the Untouched Test Set

Table 5 reports XGBoost on the 274,193-URL test set (258,096 benign, 16,097 phishing) at threshold 0.5. Test accuracy (86.89%) and AUC (0.8965) are consistent with validation (85.12%, 0.8975), confirming that the domain-disjoint validation set was a reliable proxy.

Table 5: XGBoost performance on the untouched test set (threshold 0.5).

Metric	Value	Metric	Value
Accuracy	0.8689	ROC-AUC	0.8965
Precision (phishing)	0.2820	Average precision	0.5478
Recall (phishing)	0.7979	Balanced accuracy	0.8356
F1 (phishing)	0.4167	MCC	0.4239
False-positive rate	0.1267	Specificity	0.8733
False-negative rate	0.2021	NPV	0.9858

The confusion matrix (Fig. 6a) shows 12,843 of 16,097 phishing URLs detected and 3,254 missed. The model is strongly recall-oriented: 32,704 benign URLs (12.7%) are flagged, so each true detection is accompanied by about 2.5 false alarms, giving the low precision of 0.28. Conversely, a URL predicted benign is benign 98.6% of the time (NPV). This suits a first-line filter that routes links to further checks rather than automatic blocking; Section 5.5 shows how the graded score mitigates the false-alarm load.

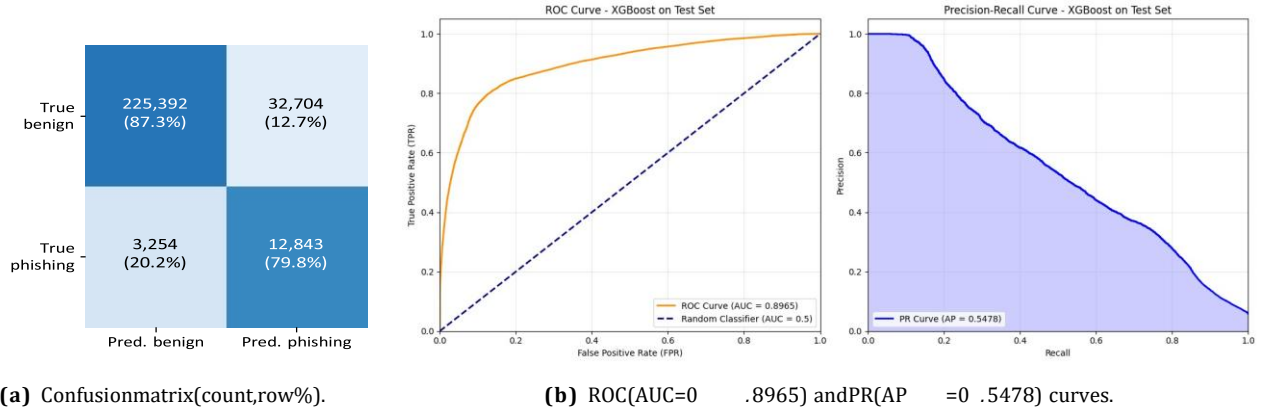


Figure 6: XGBoost on the untouched test set (274,193 URLs). Row percentages in (a) give specificity (benign row) and recall (phishing row).

Fig. 6b gives the threshold-independent view. The ROC curve shows good ranking, but the PR curve is more informative under imbalance [21]: the average precision of 0.548 is 9.3 times the 0.059 of a random classifier, yet precision, near 1.0 for the first 10% of recall, falls to about 0.6 at recall 0.4 and 0.3 at recall 0.8 before collapsing, so high recall costs many false alarms with lexical features alone.

5.4 Explainability with SHAP

Global explanations. Fig. 7 summarises SHAP on 500 test URLs. The number of path segments is by far the most influential feature (mean $|\phi| = 0.646$), followed by URL length (0.422), number of special characters (0.376), dot count (0.252), hyphen count (0.165), minimum token length (0.140), URL entropy (0.102), HTTPS (0.097), suspicious TLD (0.089) and path length (0.084). The beeswarm plot shows the direction: high path depth, length, special characters and hyphens push toward phishing, consistent with phishing kits hosted under deep, randomised paths on compromised or free hosting domains. The dependence plot (Fig. 7c) shows a sharp step: URLs with no path receive strongly negative SHAP values, while one or more segments push toward phishing. This matches how the data were built—Majestic benign URLs have no path at all—and is an early sign of the source dependence discussed in Section 5.6. SHAP also reorders the Gini ranking of Fig. 3b (path entropy drops out of the top-10), reflecting the known bias of impurity importance toward high-cardinality features.

Local explanations. Fig. 8 explains two individual test URLs. For a phishing URL scored $p = 0.830$, many special characters, path segments, length and hyphens drive the score up, partly offset by hostname length and HTTPS. For a benign URL scored $p = 0.137$, the absence of a path ($\phi = -0.72$) and few special characters dominate, outweighing small positive contributions from hyphens and entropy.

5.5 Multi-Level Risk Scoring

Table 6 and Fig. 9 show how the 274,193 test URLs fall into risk levels. The scale is meaningful: the empirical phishing rate rises monotonically from 0.9% (SAFE) through 2.3%, 6.2% and 16.1% to 44.5% (CRITICAL), i.e., CRITICAL URLs are 7.6 times more likely to be phishing than the base rate of 5.87%. HIGH and CRITICAL

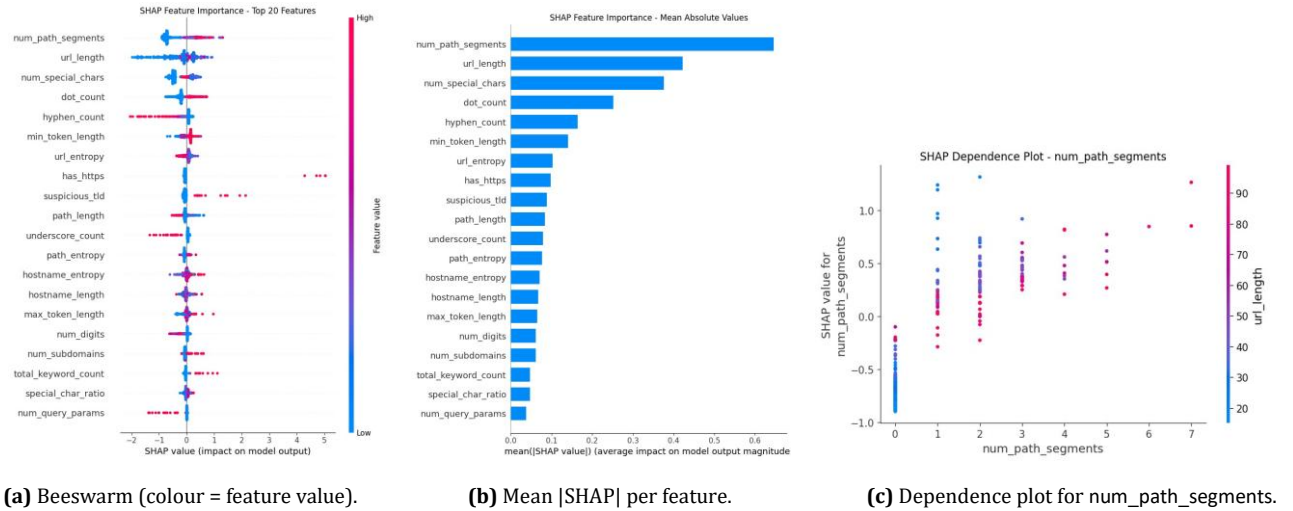
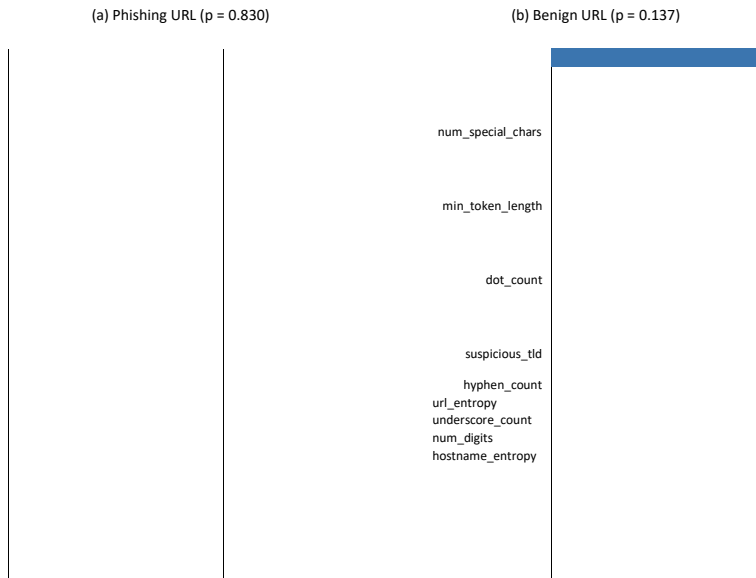


Figure 7: Global SHAP explanations for XGBoost on 500 test URLs.



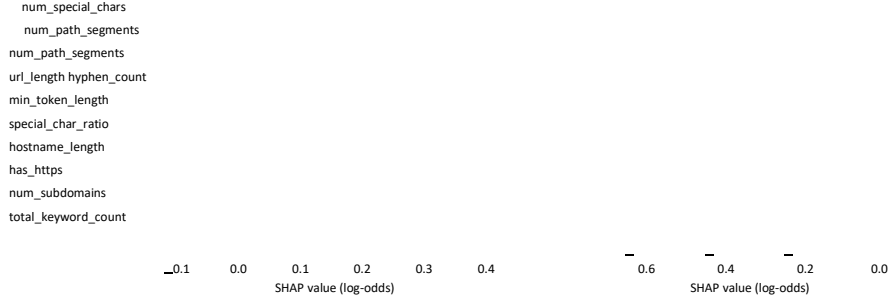


Figure 8: Local SHAP explanations (top five positive and negative contributions) for (a) a correctly detected phishing URL and (b) a correctly classified benign URL. Red pushes toward phishing, blue toward benign.

together capture 76.6% of all phishing while containing only 10.2% of benign URLs, and the SAFE level holds 61.0% of benign URLs while leaking only 8.9% of phishing. An analyst reviewing only CRITICAL alerts would inspect 21,543 URLs instead of 45,547 (all positive predictions) and nearly double precision from 0.28 to 0.45.

Scores are not calibrated: CRITICAL URLs have $p > 0.8$ but are phishing only 44.5% of the time, as expected from training at a 25% prior. Prior-shift correction or Platt/isotonic calibration on validation data would fix this.

Table 6: Risk-level distribution on the test set.

Level	Score	Total	Benign	Phishing	Phishing %
SAFE	0–20	158,943	157,517	1,426	0.9
LOW	21–40	61,983	60,548	1,435	2.3
MEDIUM	41–60	14,685	13,781	904	6.2
HIGH	61–80	17,039	14,298	2,741	16.1
CRITICAL	81–100	21,543	11,952	9,591	44.5

5.6 External Validation and Dataset Shift

We applied the identical feature extractor and the trained XGBoost model to all 235,795 PhiUSIIL URLs. The feature keyword_security was not produced by the external extraction script and was set to 0.

A label-polarity pitfall. PhiUSIIL encodes legitimate URLs as 1 and phishing as 0, the opposite of our convention. Reading label 1 as phishing gives an apparently strong result—“100% recall, 60.9% accuracy, AUC 0.657”—but this “perfect recall” actually means that every one of the 134,850 *legitimate* URLs was flagged. With the correct mapping (Table 7, Fig. 10a, left), the model predicts phishing for 226,997 of 235,795 URLs (96.3%). It detects 91.3% of phishing URLs but labels *no* legitimate URL as benign (specificity 0%). Accuracy is 39.1% and ROC-AUC is 0.344—below chance, meaning the model systematically ranks legitimate PhiUSIIL URLs as *more* suspicious than phishing ones. Fig. 10a (right) contrasts this with in-distribution performance.

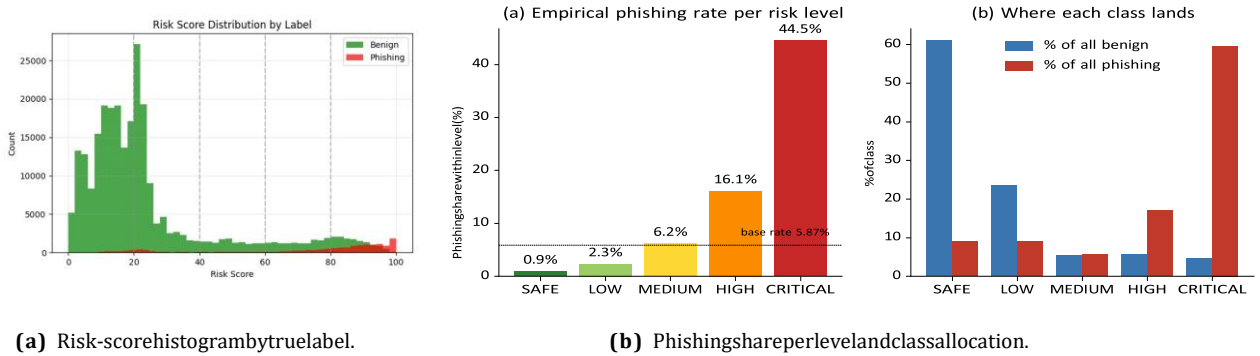
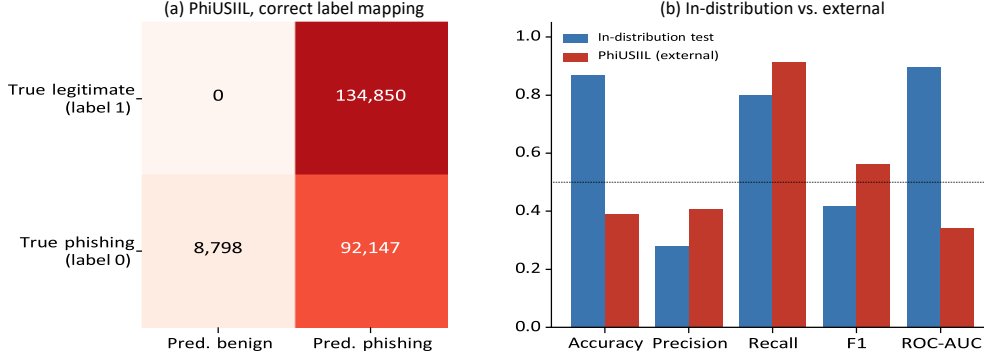


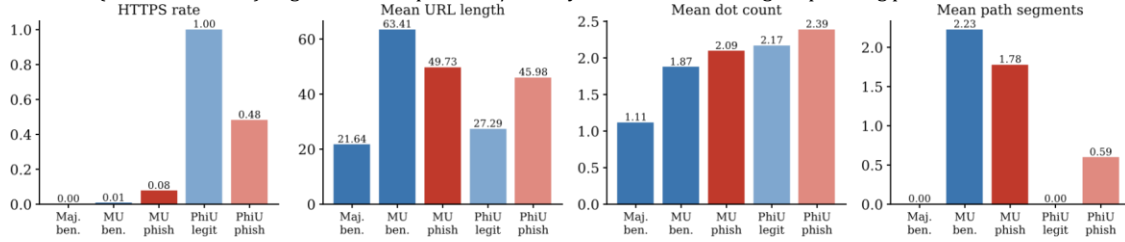
Figure 9: Multi-level risk scoring on the test set. (a) Distribution of risk scores for benign (green) and phishing (red) URLs; dashed lines are level boundaries. (b-left) Empirical phishing rate within each level versus the 5.87% base rate; (b-right) share of all benign and all phishing URLs that fall into each level.

Table 7: XGBoost on the untouched in-distribution test set versus PhiUSIIL (external) with the correct label mapping.

Metric	Test (in-distribution)	PhiUSIIL (external)
Phishing prevalence	5.9%	42.8%
Accuracy	0.8689	0.3908
Precision (phishing)	0.2820	0.4059
Recall (phishing)	0.7979	0.9128
Specificity	0.8733	0.0000
F1 (phishing)	0.4167	0.5620
Balanced accuracy	0.8356	0.4564
ROC-AUC	0.8965	0.3435



(a) Left: PhiUSIIL confusion matrix with the correct label mapping—every legitimate URL is flagged. Right: in-distribution test vs. PhiUSIIL metrics (dotted line = 0.5); higher external precision/F1 only reflect PhiUSIIL’s higher phishing prevalence.



(b) Feature means across training sources (blue/red) and PhiUSIIL (light blue/light red): HTTPS usage and dot count are inverted between training-benign and PhiUSIIL-legitimate URLs.

Figure 10: External validation on PhiUSIIL and the source artefacts that explain its failure.

Why the model fails: source artefacts. To diagnose the collapse we re-ran the feature extractor on 5,000 random URLs from each source group (Fig. 10b). Three artefacts stand out:

1. **HTTPS.** No Majestic URL and only 0.6% of Malicious-dataset benign URLs carry an https scheme, versus 7.6% of training phishing URLs. The model therefore learns “HTTPS $\Rightarrow$ phishing”. In PhiUSIIL, 100% of legitimate URLs use HTTPS, compared with 48% of phishing URLs.
2. **www prefix.** Training URLs were normalised with www. removed, but the external extraction used a simpler normaliser that kept it. Every legitimate PhiUSIIL URL therefore gains a dot and a subdomain: mean dot count is 2.17 versus 1.12 for Majestic, and dot_count is the fourth most important SHAP feature. This is a train–serve skew in our own pipeline.
3. **Bare domains as the benign prototype.** About 69% of training benign URLs come from Majestic, which lists bare domains (mean length 21.6, zero path segments). The model thus treats “short, no path, no scheme” as benign—the step in Fig. 7c—a pattern that does not describe real legitimate URLs encountered in browsing.

The first and third are covariate shift arising from how each dataset was collected [18]; together they are a textbook case of shortcut learning [7]. The in-distribution result is valid for the distribution it measures, but that distribution differs systematically from the real-world one.

5.7 Discussion

Three conclusions follow. **(i) Leakage control works.** With domain-disjoint splits, validation and test agree closely (AUC 0.8975 vs. 0.8965) and scores are well below the >95% often reported with random splits. **(ii)**

Data matters more than the learner. The four ensembles differ by under 0.003 AUC; LR’s lower scores reflect high bias and RF’s larger gap its deeper trees. **(iii) A small gap is necessary but not sufficient.** XGBoost has a low gap yet fails on PhiUSIIL because all development splits share the same source artefacts. SHAP was essential for finding this: the top feature effectively encodes “has a path”, which pointed to the bare-domain benign data. Explainability thus served to *audit* the model, not only to justify it. Operationally, treating CRITICAL as “warn” and HIGH as “review” captures three quarters of phishing at manageable load, but the model should be retrained on realistic benign URLs before real deployment.

6 Conclusion

We built a URL-only phishing detection and risk scoring system with a leakage-aware evaluation protocol, a five-model bias–variance comparison and SHAP explanations. On a domain-disjoint, naturally imbalanced test set of 274,193 URLs, XGBoost achieved 86.89% accuracy, 0.8965 ROC-AUC and 79.79% phishing recall with a train–validation accuracy gap of only 2.1 points, and its five risk levels were monotone in observed phishing rate. External validation on PhiUSIIL, after correcting its inverted label convention, showed that these results do not transfer: the model flagged 96.3% of all URLs and reached a below-chance AUC of 0.344. Feature-level analysis attributed this failure to source artefacts (HTTPS, www and bare-domain benign URLs) and to a normalisation mismatch.

Limitations. (1) Benign and phishing URLs come from different sources, so part of the in-distribution performance reflects source identity. (2) Only lexical features are used (no content, WHOIS, DNS or certificate data). (3) External extraction used a different normaliser and lacked one keyword feature. (4) Domains are the last two host labels, which merges sites under suffixes such as co.uk; the suspicious-TLD check matches substrings. (5) Scores are uncalibrated, the threshold is fixed at 0.5, SHAP used 500 samples, and no hyperparameter search or multi-seed confidence intervals were run.

Future work. We plan to (i) rebuild the benign set from full URLs (e.g., crawled pages of Tranco-ranked sites [24]) and remove or randomise scheme and www artefacts during training, (ii) enforce a single shared preprocessing module for training and inference and use the Public Suffix List for domain extraction, (iii) evaluate with leave-one-source-out splits and domain adaptation, (iv) calibrate scores and tune thresholds on validation data for a target false-alarm budget, and (v) add host-based and content features and compare against character-level deep models.

Reproducibility. All code, the trained model, the list of the 47 selected features, dataset download instructions, a requirements.txt and a README with step-by-step instructions are available at <https://github.com/YOUR-USERNAME/phishing-url-risk-scoring>. All random operations use seed 42.

References

- [1] Anti-Phishing Working Group, “Phishing Activity Trends Reports,” APWG, 2023–2025. [Online]. Available: <https://apwg.org/trendsreports/>
- [2] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, “Beyond blacklists: Learning to detect malicious web sites from suspicious URLs,” in *Proc. 15th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2009, pp. 1245–1254.
- [3] D. Sahoo, C. Liu, and S. C. H. Hoi, “Malicious URL detection using machine learning: A survey,” *arXiv preprint arXiv:1701.07179*, 2017.
- [4] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, “Machine learning based phishing detection from URLs,” *Expert Systems with Applications*, vol. 117, pp. 345–357, 2019.
- [5] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, “Leakage in data mining: Formulation, detection, and avoidance,” *ACM Trans. Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012.
- [6] A. Hannousse and S. Yahiouche, “Towards benchmark datasets for machine learning based website phishing detection: An experimental study,” *Engineering Applications of Artificial Intelligence*, vol. 104, p. 104347, 2021.
- [7] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann, “Shortcut learning in deep neural networks,” *Nature Machine Intelligence*, vol. 2, pp. 665–673, 2020.

- [8] S. Garera, N. Provos, M. Chew, and A. D. Rubin, "A framework for detection and measurement of phishing attacks," in *Proc. ACM Workshop on Recurring Malcode (WORM)*, 2007, pp. 1–8.
- [9] R. Verma and A. Das, "What's in a URL: Fast feature extraction and malicious URL detection," in *Proc. 3rd ACM Int. Workshop on Security and Privacy Analytics (IWSPA)*, 2017, pp. 55–63.
- [10] H. Le, Q. Pham, D. Sahoo, and S. C. H. Hoi, "URLNet: Learning a URL representation with deep learning for malicious URL detection," *arXiv preprint arXiv:1802.03162*, 2018.
- [11] A. Aljofey, Q. Jiang, Q. Qu, M. Huang, and J.-P. Niyigena, "An effective phishing detection model based on character level convolutional neural network from URL," *Electronics*, vol. 9, no. 9, p. 1514, 2020.
- [12] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [13] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [14] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems*, vol. 31, 2018.
- [15] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [16] A. Prasad and S. Chandra, "PhiUSIL: A diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning," *Computers & Security*, vol. 136, p. 103545, 2024.
- [17] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [18] J. G. Moreno-Torres, T. Raeder, R. Alaiz-Rodríguez, N. V. Chawla, and F. Herrera, "A unifying view on dataset shift in classification," *Pattern Recognition*, vol. 45, no. 1, pp. 521–530, 2012.
- [19] Majestic, "The Majestic Million," [Online]. Available: <https://majestic.com/reports/majestic-million>
- [20] M. Siddhartha, "Malicious URLs dataset," Kaggle, 2021. [Online]. Available: <https://www.kaggle.com/datasets/sid321axn/malicious-urls-dataset>
- [21] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, p. e0118432, 2015.
- [22] D. Chicco and G. Jurman, "The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation," *BMC Genomics*, vol. 21, p. 6, 2020.
- [23] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [24] V. Le Pochat, T. Van Goethem, S. Tajalizadegan, M. Korczynski, and W. Joosen, "Tranco: A research-oriented top sites ranking hardened against manipulation," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2019.